

## **Task #2: Project Proposal V0**

At last, it's time to really think about your project.

You will create 2 project proposals, one for your main idea and one for a backup idea. You can be flexible with what you include in your proposal. However, keep in mind that the more information you include, the better feedback you will receive from us.

Here is what each project proposal should roughly look like:

1. Project title and short description: what will your project do?
2. Technical description: What core functionalities does your project include? How would you roughly go about designing each part of your project?
3. I/Os: What and how many I/Os will your project have? Note that you will be limited to 12 inputs and 12 outputs.
4. List of external hardware peripherals you might need (if applicable)
5. Module Header: This is optional, but having a module header in mind can prove useful in nailing down the specifications of your project. Note that you will need a complete module header in subsequent iterations of this proposal.

# Tiny Memory Controller - Main

I want to implement a small memory controller that accepts read/write requests, schedules the accesses, and communicates with the external memory using a simplified protocol. It will explore the ideas of arbitration, latency, and throughput tradeoffs with the limited amount of area.

**Technical description: What core functionalities does your project include? How would you roughly go about designing each part of your project?**

The four separate modules of my project will include:

- Request Queue (will include some FIFOs and registers to store all requests by the fake CPU)
- Scheduler/Arbiter that will control which requests are served first or later (ex. A read request will probably be prioritized over a write request)
- Memory Interface FSM that will actually spit out the commands to the DRAM (activate, read/write, wait, respond, idle)
- Fake memory module

One thing to include is fake memory delays (read latency, write latency, turnaround time R->W or W->R)

To test my memory controller, I can have back to back reads, back to back writes, read after writes, etc and show the latency, throughput, and data.

**I/Os: What and how many I/Os will your project have? Note that you will be limited to 12 inputs and 12 outputs.**

Input pins:

|                     |                                                                         |
|---------------------|-------------------------------------------------------------------------|
| Input pins [7:0]    | If pin 2 = 0, [3:0] used as address<br>If pin 2 = 1, [7:0] used as data |
| Bidirectional pin 0 | req_valid                                                               |
| Bidirectional pin 1 | req_rw                                                                  |
| Bidirectional pin 2 | Req_data_phase (0=request address phase, 1=w_data phase)                |

## Output pins

|                     |                                                                                                                                                                                                                                 |
|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| output pins [7:0]   | resp_rdata                                                                                                                                                                                                                      |
| Bidirectional pin 3 | resp_valid                                                                                                                                                                                                                      |
| Bidirectional pin 4 | resp_busy                                                                                                                                                                                                                       |
| Bidirectional pin 5 | Resp_is_read: tells whether current response outputted by memory controller is a read or write req. If read, resp_valid means data is ready to read from resp_rdata, if write, resp_valid means data has been written to memory |

## List of external hardware peripherals you might need (if applicable)

No external hardware peripherals are necessary for now. For testing and debugging purposes, I can include LEDs for my status bits (resp\_busy, resp\_valid) or my input address. I can also add 7 segment displays for my resp\_rdata (if I write numbers as my testing data). However, these hardware peripherals are not necessary (it's just for debug and demo purposes).

**Module Header: This is optional, but having a module header in mind can prove useful in nailing down the specifications of your project. Note that you will need a complete module header in subsequent iterations of this proposal.**

memctrl

```
 #(      .addr_width()
        .data_width(8),
        .request_q_size(2),
        .read_latency(3),
        .write_latency(2),
        .turnaround_time(1))
 (      .clk (clk),
        .rst_n (rst_n),

        .req_valid (req_valid),
        .req_rw (req_rw),
        .req_phase (req_phase),
        .req_payload(req_payload),

        .resp_valid (resp_valid),
        .resp_rdata (resp_rdata),
```

```
.busy (busy)  
);
```

# Mini Weight Stationary Systolic Array - Side 1

I want to create a mini matrix multiply unit that will stream in a mini matrix as activation, multiply it with the weights in the register of each processing element, and create an output matrix after a calculated amount of clock cycles. Although I feel most passionate about this topic, my main concern for this project is that it will take up way too much area.

## **Technical description: What core functionalities does your project include? How would you roughly go about designing each part of your project?**

Processing Elements: a multiply accumulate unit with registers for the stationary weights and accumulation

Streaming activation interface: activations are streamed into the processing elements every cycle (shift register)

Need a lot of control signals to signal the loading of the weights, the start of the calculation, and finish. Need a counting mechanism to evaluate when the calculation will be done.

## **I/Os: What and how many I/Os will your project have? Note that you will be limited to 12 inputs and 12 outputs.**

I am actually quite unsure how I want to structure the I/O for this project. If I already have the weights and activation hardcoded to use in my modules, I don't really need any other inputs besides a start button. However, if I want to input weights and activation during runtime, I need input bits for the data as well as the mode (load weights, load activation, calculate)

As for the output pins, I would need a data\_valid, and some kind of a data output. If I only do 2\*2 matrix multiply, I would get 4 outputs max, and so I should limit my output data bitwidth to [2:0] and have them come out of my output [2:0], [5:3], and bidirectional [2:0], [5:0].

I will need to have control logic for manipulating the bidirectional ports.

I will have to think about this section more (especially how to limit the size of my matrix and data).

## **List of external hardware peripherals you might need (if applicable)**

N/A

**Module Header: This is optional, but having a module header in mind can prove useful in nailing down the specifications of your project. Note that you will need a complete module header in subsequent iterations of this proposal.**

start (pulse to begin calculation or load)

mode [1:0] (load weight, load activation, or calculate) – only if I decide to input weights and activation during runtime

Data\_in [idk how many bits yet]

Data\_valid

data\_out\_one, data\_out\_two, data\_out\_three, data\_out\_four (not sure how many bits yet)

A major concern is that the data\_out should probably be greater than 3 bits since it's a multiplied value. Maybe I could store the outputs in a register and output them slowly one by one through the output pins once the calculations are done. Same goes for the input data. I'm not sure how I will input all the weights and activations (if it isn't hardcoded) with the limited number of input pins. Do I just assign data (weights) to 1 processing element each cycle, putting the weight into the corresponding PE using a counter(ex. Processing element 1 gets input data weight at cycle 1)? This wouldn't work for the activation streaming because it would need to stream multiple values each cycle. I can also fill in FIFOs beforehand during the activation fill mode, and start the systolic array activation streaming with FIFOs.

# Maze Runner - Side 2

It's a simple VGA game where the player has to navigate through a hardcoded maze. It will use VGA to have the players play the game on a screen. The mazes will be hardcoded and I plan to have at least 2 different levels of the maze.

**Technical description: What core functionalities does your project include? How would you roughly go about designing each part of your project?**

There will be several pages, from level introduction page to play page, from play page to win page if the player reaches exit cell (or from play to game over if timer expires), win page to next level page if the player still has mazes to go, and then game over page if the player completes all mazes prepared for them.

For the mazes, I am planning to hardcode them, 0 representing a path and 1 representing a wall. I can represent each row of the maze as a bit vector.

I can store the player position in registers, and on a move, I can calculate the output position, check if it's a valid cell (within grids, not a wall), and update the player position.

The graphics are output via VGA with hsync, vsync, and an RGB width.

There will be a timer for the time limit within the game (timer displayed on the screen is also planned for now).

**I/Os: What and how many I/Os will your project have? Note that you will be limited to 12 inputs and 12 outputs.**

|                                                                                                                            |                                                               |
|----------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| Input pins [0]<br>Input pins [1]<br>Input pins [2]<br>Input pins [3]<br>Input pins [4]<br>Input pins [5]<br>Input pins [6] | Up<br>Down<br>Left<br>Right<br>Start<br>Reset<br>Select Level |
| Output pins[0]<br>Output pins[1]<br>Output pins[2]<br>Output pins[3]<br>Output pins[4]                                     | Hync<br>Vsync<br>Vga_r<br>Vga_g<br>vga_b                      |

**List of external hardware peripherals you might need (if applicable)**

VGA display, buttons/switches for up/down/left/right

**Module Header: This is optional, but having a module header in mind can prove useful in nailing down the specifications of your project. Note that you will need a complete module header in subsequent iterations of this proposal.**

```
 #( grid_width = 16, grid_height = 12, cell_size = 8, levels = 2)
 btn_up = in_pins[0];
 btn_down = in_pins[1];
 btn_left = in_pins[2];
 btn_right = in_pins[3];
 btn_start = in_pins[4];
 btn_reset = in_pins[5];
 maze_select = in_pins[7:6]
```

Output pins → all mapped to vga related outputs (vsync, hsync, etc)